

Mini-projets — Systèmes Distribués & GPU

1. Accélération d'algorithmes classiques avec OpenMP

- Implémenter : tri (merge sort, quicksort), multiplication de matrices
- Comparer version séquentielle vs parallèle
- Mesurer : speedup, efficacité (loi d'Amdahl)

2. Système de traitement parallèle d'images

- Appliquer filtres (flou, détection de contours) en parallèle
- Utiliser threads ou OpenMP
- Comparaison CPU multi-thread vs GPU

3. Système de tri distribué (Distributed Sort)

- Chaque nœud trie une partie
- Fusion finale (merge distribué)
- Optimiser les communications réseau

4. Mini moteur MapReduce avec MPI

- Implémenter Map, Shuffle, Reduce
- Comparaison avec Apache Spark

5. Détection d'objets simple sur GPU

- Traitement d'image parallèle
- Exemple : détection de contours (Sobel)
- Intégration avec OpenCV

6. Entraînement distribué d'un modèle ML

- Réseau de neurones simple
- Répartition des données

7. Comparaison Data Parallelism vs Model Parallelism

- Implémenter les deux approches
- Analyser performance et scalabilité

8. Détection rapide d'anomalies dans des images médicales

Application : aide au diagnostic (radiologie) | Technos : GPU (CUDA/OpenCL), parallélisme d'images

- Traitement parallèle d'images (scanner, IRM)
- Détection de contours / zones suspectes

9. Analyse du trafic routier en temps réel

Application : gestion intelligente de la circulation

- Traitement de flux vidéo ou données capteurs
- Analyse distribuée
- Détection d'embouteillages

10. Analyse vidéo distribuée avec accélération GPU

Application : vidéosurveillance intelligente

- Chaque nœud traite une portion du flux vidéo
- Traitement d'image accéléré sur GPU
- Agrégation des résultats via MPI

11. Simulation météo distribuée avec GPU

Application : prévision climatique

- Partition de la grille météo entre nœuds
- Calcul parallèle GPU pour chaque zone
- Communication des frontières (MPI)

12. Système de détection d'intrusion temps réel (GPU + streaming)

Application : cybersécurité

- Analyse de flux réseau en temps réel
- Détection parallèle (GPU)
- Distribution des flux entre nœuds

13. Entraînement distribué de deep learning (GPU)

Application : IA

- Data parallelism sur plusieurs GPU
- Synchronisation des gradients (MPI ou framework)

14. Système de recherche distribué accéléré GPU

Application : moteur de recherche

- Index distribué
- Recherche parallèle GPU (matching rapide)

15. Système de détection de fraude distribué avec Blockchain

Application : finance / paiements sécurisés

- Les transactions sont distribuées sur plusieurs nœuds
- Détection de fraude accélérée sur GPU
- Stockage sécurisé dans une blockchain

16. Système de vote électronique sécurisé et distribué

Application : e-voting

- Votes enregistrés dans une blockchain
- Validation distribuée
- Vérification rapide sur GPU (hashing massif)

17. Détection de Maladies des Cultures

- Affiner ResNet50 sur PlantVillage pour classifier 38 catégories de maladies de plantes à partir de photos de feuilles
- évaluer avec une matrice de confusion.

Dataset : Dataset PlantVillage (54 000+ images, 38 classes) — Kaggle / HuggingFace

- Appliquer le transfert learning à l'agriculture
- gérer l'augmentation de données
- identifier les paires de maladies les plus confondues via la matrice de confusion.

18. Prédiction du Rendement Agricole

- Construire un pipeline ML accéléré par GPU avec RAPIDS cuML pour prédire le rendement des cultures à partir des données historiques FAO
- expliquer les prédictions avec SHAP.

Dataset : Dataset FAO FAOSTAT rendements agricoles (28 000+ lignes, pays × années)

- Utiliser cuML Random Forest, XGBoost et Régression Linéaire sur GPU
- évaluer avec RMSE & R²
- générer des graphiques d'importance des features SHAP.

19. Segmentation d'Images Satellites pour la Cartographie Agricole

- Entraîner un U-Net pour classifier l'usage des terres (cultures, forêt, urbain...) pixel par pixel à partir de patches Sentinel-2
- calculer le NDVI sur GPU.

Dataset : Dataset EuroSAT (27 000 patches Sentinel-2, 10 classes d'usage des terres) — torchvision / HuggingFace

- Comprendre la segmentation sémantique et l'architecture U-Net
- évaluer avec l'IoU par classe
- cartographier optionnellement une région agricole marocaine.

20. Détection d'Invasion de Criquets Pèlerins

- Construire un pipeline d'alerte précoce bout-en-bout prédisant les conditions favorables aux criquets à partir de données satellitaires (NDVI, température, humidité du sol).

Dataset : CSV présence criquets FAO DLIS (1985–2023) ; données satellitaires Google Earth Engine (NDVI Sentinel-2, température MODIS, humidité SMAP)

- Gérer le déséquilibre des classes (SMOTE)
- entraîner des classifieurs cuML
- évaluer avec F1/ROC-AUC
- générer un graphique SHAP et une carte de chaleur des prédictions sur le Maroc.

21. TextBlob vs VADER vs FinBERT sur les actualités financières

Application : analyse de sentiment financier | Technos : FinBERT, TextBlob, VADER, Streamlit

- Comparer TextBlob, VADER et FinBERT sur Financial PhraseBank et Kaggle Financial News.
- Exécuter FinBERT sur GPU, TextBlob et VADER sur CPU comme référence.
- Récupérer des actualités via yfinance pour tester les modèles sur données réelles.
- Construire un dashboard Streamlit visualisant les scores de sentiment par titre boursier.
- Produire un tableau comparatif de précision et un notebook annoté.
- Accélération GPU : inférence FinBERT batch — gain 20–30× vs CPU.

22. Prédiction LSTM avec vs sans Sentiment

Application : prédiction de cours boursiers | Technos : LSTM, FinBERT, RSI, MACD

- Entraîner deux modèles LSTM sur GPU : prix OHLCV seuls vs prix + sentiment FinBERT.
- Intégrer des indicateurs techniques (RSI, MACD) comme features supplémentaires.
- Réaliser un backtesting sur 2 ans sur 5 actions.
- Comparer Modèle A (prix seul) vs Modèle B (prix + sentiment + indicateurs).
- Visualiser les prédictions et calculer MAE/RMSE sur la période de test.
- Accélération GPU : entraînement LSTM-Attention 20–40× plus rapide vs CPU.

23. Sentiment Audio & Vidéo Politique — La voix de Powell influence-t-elle les marchés ?

Application : analyse multimodale des discours FOMC | Technos : FinBERT, Wav2Vec2, DeepFace

- Analyser les discours FOMC via trois canaux : texte (FinBERT), audio (Wav2Vec2), vidéo (DeepFace) — tous sur GPU.
- Tester quelle combinaison prédit le mieux les mouvements du S&P 500.
- Comparer texte seul vs audio vs fusion multimodale.
- Corréler les scores de sentiment avec les variations du S&P 500 après chaque conférence FOMC.
- Produire un rapport comparatif avec graphiques de corrélation sentiment–marché.
- Accélération GPU : trois modèles (Wav2Vec2 + FinBERT + DeepFace) dans le même pipeline.

24. Parallélisation du tri rapide avec OpenMP

- Implémenter quicksort parallèle avec OpenMP
- Comparer version séquentielle vs parallèle
- Tester avec des tableaux de 10K, 100K et 1M d'éléments
- Mesurer le speedup selon le nombre de threads et identifier le point de saturation

25. Multiplication de matrices sur GPU

- Implémenter la multiplication de matrices sur GPU (CuPy ou CUDA)
- Comparer les temps d'exécution CPU vs GPU

26. Filtrage d'images en parallèle (OpenMP / GPU)

- Appliquer des filtres (flou gaussien, Sobel) sur un batch d'images
- Paralléliser avec OpenMP ou GPU (OpenCV-CUDA / CuPy)

27. Simulation de diffusion thermique parallèle

- Résoudre l'équation de la chaleur sur une grille 2D (méthode d'Euler explicite)
- Paralléliser avec OpenMP ou CUDA — chaque cellule calculée par un thread
- Tester sur des grilles 256x256, 512x512 et 1024x1024
- Visualiser l'évolution de la température et mesurer le speedup

28. Comptage de mots distribué avec MPI

- Distribuer un corpus textuel entre N nœuds MPI
- Compter les fréquences localement puis fusionner avec MPI_Reduce
- Comparer avec une implémentation séquentielle

29. Benchmark CPU vs GPU sur des opérations vectorielles

- Mesurer les performances de somme, produit scalaire, norme sur CPU et GPU
- Tester différentes tailles de vecteurs (1K à 100M éléments)

30. Pipeline de traitement de données distribué avec MPI

- Distribuer un dataset CSV entre plusieurs nœuds
- Effectuer des agrégations locales puis une réduction globale

31. Classification d'images simple sur GPU

- Entraîner un CNN (3-4 couches) sur CIFAR-10 avec PyTorch
- Comparer le temps d'entraînement CPU vs GPU par epoch

32. Clustering K-Means parallèle

- Implémenter K-Means sur GPU avec cuML (RAPIDS)
- Comparer avec la version CPU scikit-learn
- Tester sur 10K, 100K et 1M points pour différentes valeurs de K
- Visualiser les clusters et mesurer le speedup selon N et K

33. Système de fichiers distribué simple

- Implémenter un Name Node et N Data Nodes communiquant via sockets Python
- Gérer les opérations PUT / GET avec découpage en blocs et réplication (facteur 2)

34. Détection de spam avec SVM accéléré GPU

- Entraîner un SVM sur le dataset SMS Spam Collection (5 574 messages)
- Comparer sklearn SVM (CPU) vs cuML SVM (GPU) avec kernels linéaire et RBF
- Analyser les faux positifs (emails légitimes classés spam)
- Comparer vitesse d'entraînement et précision/rappel/F1

35. Simulation N-corps sur GPU

- Simuler les interactions gravitationnelles entre N particules (algorithme $O(N^2)$)
- Paralléliser sur GPU — chaque thread calcule les forces sur une particule
- Tester pour N = 1 000, 5 000 et 10 000 particules
- Visualiser les trajectoires et mesurer le speedup vs CPU

36. Analyse de sentiment distribuée

- Distribuer un corpus de 100K+ avis entre N processus
- Utiliser DistilBERT sur GPU par nœud, VADER sur CPU comme référence
- Agréger les scores et calculer la distribution par catégorie
- Comparer précision et throughput (avis/seconde) selon le nombre de nœuds

37. Détection d'anomalies dans des données financières

- Utiliser le dataset Kaggle Credit Card Fraud (284K transactions, 0.17% de fraudes)
- Appliquer Isolation Forest et un Autoencoder PyTorch sur GPU

38. Rendu parallèle par lancé de rayons

- Implémenter un ray tracer basique (sphères, ombres, réflexions) en Python
- Paralléliser avec multiprocessing (CPU) puis Numba/CUDA (GPU)
- Tester sur des résolutions 400x300, 800x600 et 1920x1080
- Mesurer les FPS et analyser l'impact de la profondeur de récursion

39. Compression d'images parallèle avec DCT

- Diviser les images en blocs 8x8 et appliquer la DCT-2D (standard JPEG)
- Paralléliser le traitement des blocs avec OpenMP ou CuPy
- Tester différents niveaux de quantification

40. Calcul de PageRank distribué

- Implémenter PageRank par la méthode des puissances itératives avec MPI
- Distribuer les noeuds du graphe — chaque processus gère un bloc de lignes

41. Prédiction de séries temporelles sur GPU

- Entraîner un LSTM avec PyTorch sur un dataset temporel réel (UCI, yfinance...)
- Utiliser une fenêtre glissante de 30 pas pour prédire les 7 suivants

42. Simulation Monte Carlo parallèle

- Estimer Pi et pricer une option européenne par simulation de trajectoires
- Paralléliser avec GPU (CuPy random) ou multiprocessing CPU

43. Transcription audio distribuée

- Distribuer un corpus de fichiers audio entre N processus avec accès GPU
- Utiliser Whisper (tiny, base, small) ou Wav2Vec2 pour la transcription

44. Détection d'objets en temps réel avec GPU

- Utiliser YOLOv8n sur un flux vidéo (webcam ou vidéo de test)
- Mesurer les FPS avec et sans GPU selon la résolution d'entrée

45. Résolution distribuée d'équations linéaires

- Implémenter la méthode de Jacobi distribuée avec MPI sur un système $Ax=b$
- Chaque processus détient un bloc de lignes et échange les valeurs de bordure
- Tester sur un système issu de la discrétisation de l'équation de Poisson
- Comparer convergence Jacobi vs Gauss-Seidel vs solution directe numpy

46. Analyse de logs réseau distribuée

- Distribuer l'analyse de logs entre plusieurs nœuds MPI
- Détecter des patterns suspects : scan de ports, brute force, DDoS
- Implémenter une détection par règles ET par seuil statistique (z-score)
- Agréger les résultats et mesurer le throughput (logs/seconde) selon le nombre de nœuds

47. Amélioration d'images sur GPU

- Appliquer filtre bilatéral, débruitage par ondelettes et autoencoder débruiteur
- Tester sur des images avec bruit gaussien ajouté artificiellement

48. Traitement de flux de données en temps réel

- Simuler un producteur de données IoT (mesures toutes les 10ms) avec ZeroMQ
- Appliquer des transformations sur GPU par mini-batches côté consommateur
- Observer le compromis latence vs throughput selon la taille du mini-batch
- Mesurer le taux de remplissage de la queue sous différentes charges (backpressure)

49. Calcul de matrices de similarité sur GPU

- Calculer la similarité cosinus entre N vecteurs d'embeddings (Word2Vec, SBERT)
- Implémenter en NumPy (CPU) et CuPy/PyTorch (GPU)

50. Simulation de réseau de neurones simple sur GPU

- Implémenter un MLP from scratch en NumPy : forward, backward, gradient descent
- Réécrire les opérations matricielles clés en CuPy ou CUDA
- Entraîner sur MNIST et comparer le temps par epoch CPU vs GPU
- Visualiser la convergence et vérifier que les résultats sont identiques aux deux versions

51. Reconnaissance de gestes avec GPU

- Capturer des gestes via webcam avec OpenCV (ou utiliser le dataset Jester)
- Classifier avec un CNN léger (MobileNet) entraîné sur GPU
- Assurer une inférence en temps réel (<50ms par frame)
- Déployer une démo fonctionnelle et analyser les gestes les plus souvent confondus

52. Moteur de recherche distribué simple

- Construire un index inversé sur 10K+ documents et le distribuer entre N nœuds
- Pour une requête multi-mots, interroger les nœuds en parallèle et scorer avec TF-IDF
- Analyser l'équilibre de charge selon la stratégie de partitionnement
- Comparer le temps de réponse séquentiel vs distribué sur 20 requêtes test

53. Génération de texte simple sur GPU

- Utiliser GPT-2 small via HuggingFace et comparer les temps CPU vs GPU
- Tester la génération de séquences de 50, 200 et 500 tokens
- Comparer trois stratégies de décodage : greedy, beam search (k=5), top-p sampling
- Analyser qualitativement les différences de cohérence entre stratégies

54. Simulation physique de particules sur GPU

- Simuler des collisions élastiques en 2D avec position, vitesse et masse
- Paralléliser la détection de collision (force brute $O(N^2)$) sur GPU avec Numba/CuPy
- Tester pour 1K, 10K et 100K particules

55. Classification médicale d'images sur GPU

- Fine-tuner ResNet18 ou EfficientNet-B0 sur un dataset médical (Chest X-Ray, HAM10000...)
- Évaluer avec précision, rappel et F1 par classe

56. Traduction automatique distribuée

- Distribuer un corpus de 10K phrases entre N processus GPU
- Utiliser Helsinki-NLP/opus-mt (modèle Marian) via HuggingFace
- Évaluer la qualité avec le score BLEU sur un jeu de test annoté

57. Prédiction de consommation énergétique sur GPU

- Utiliser le dataset UCI Household Electric Power Consumption (2M mesures)
- Comparer MLP, LSTM et TCN entraînés sur GPU
- Intégrer des features contextuelles : heure, jour de la semaine, température
- Évaluer avec MAE et RMSE, analyser les erreurs sur les jours atypiques

58. Simulation de trafic distribué

- Modéliser un réseau routier en graphe et distribuer les zones entre N processus MPI
- Simuler les véhicules avec le modèle car-following IDM
- Implémenter la migration de véhicules entre zones (transfert de particules)